Souvenirs

Tekst zadatka doslovno je preveden iz izvorne verzije. Čitanje nije preporučeno.

Amaru kupuje suvenire u stranoj trgovini. Postoji N **vrsta** suvenira. U trgovini je dostupno beskonačno mnogo suvenira svake vrste.

Svaka vrsta suvenira ima fiksnu cijenu. Naime, suvenir tipa i ($0 \leq i < N$) ima cijenu od $P[i]$ kovanica, gdje je $P[i]$ pozitivan cijeli broj.

Amaru zna da su vrste suvenira sortirane u silaznom redoslijedu prema cijeni, i da su cijene suvenira različite. Točnije, $P[0] > P[1] > \cdots > P[N-1] > 0$. Štoviše, uspio je saznati vrijednost $P[0]$. Nažalost, Amaru nema nikakve druge informacije o cijenama suvenira.

Kako bi kupio suvenire, Amaru će obaviti niz transakcija s prodavateljem.

Svaka transakcija sastoji se od sljedećih koraka:

1. Amaru predaje određeni (pozitivan) broj novčića prodavatelju.
2. Prodavač stavlja ove novčiće na hrpu na stol u stražnjoj sobi, gdje ih Amaru ne može vidjeti.
3. Prodavatelj razmatra svaki tip suvenira $0, 1, \ldots, N-1$ tim redom, jedan po jedan. Svaka vrsta se uzima u obzir **točno jednom** po transakciji.
    - Prilikom razmatranja suvenira tipa i, ako je trenutni broj novčića u hrpi barem $P[i]$, tada
        - prodavatelj uklanja $P[i]$ kovanica s hrpe
        - prodavač stavlja jedan suvenir tipa i na stol.
4. Prodavač daje Amaruu sve preostale kovanice u hrpi i sve suvenire na stolu.

Imajte na umu da na stolu nema kovanica ili suvenira prije početka transakcije.

Vaš je zadatak uputiti Amarua da izvrši određeni broj transakcija, tako da:

- u svakoj transakciji kupuje **barem jedan** suvenir
- ukupno kupuje **točno** i suvenira tipa i, za svaki i takav da je $0 \leq i < N$. Imajte na umu da to znači da Amaru ne bi trebao kupiti nikakav suvenir tipa 0.

Amaru ne mora minimizirati broj transakcija i ima neograničenu zalihu kovanica.

Implementation Details

You should implement the following procedure.

```
void buy_souvenirs(int N, long long P0)
```

- N: the number of souvenir types.
- $P0$: the value of $P[0]$.
- This procedure is called exactly once for each test case.

The above procedure can make calls to the following procedure to instruct Amaru to perform a transaction:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M: the number of coins handed to the seller by Amaru.
- The procedure returns a pair. The first element of the pair is an array L, containing the types of souvenirs that have been bought (in increasing order). The second element is an integer R, the number of coins returned to Amaru after the transaction.
- It is required that $P[0] > M \geq P[N-1]$. The condition $P[0] > M$ ensures that Amaru does not buy any souvenir of type 0, and $M \geq P[N-1]$ ensures that Amaru buys at least one souvenir. If these conditions are not met, your solution will receive the verdict `Output isn't correct: Invalid argument`. Note that contrary to $P[0]$, the value of $P[N-1]$ is not provided in the input.
- The procedure can be called at most 5000 times in each test case.

The behavior of the grader is **not adaptive**. This means that the sequence of prices P is fixed before `buy_souvenirs` is called.

Constraints

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ for each i such that $0 \leq i < N$.
- $P[i] > P[i+1]$ for each i such that $0 \leq i < N-1$.

Subtasks

Subtask	Score	Additional Constraints
1	4	$N = 2$
2	3	$P[i] = N - i$ for each i such that $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ for each i such that $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ for each i such that $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ for each i such that $0 \leq i < N - 1$.
6	33	No additional constraints.

Example

Consider the following call.

```
buy_souvenirs(3, 4)
```

There are $N = 3$ types of souvenirs and $P[0] = 4$. Observe that there are only three possible sequences of prices P: $[4, 3, 2]$, $[4, 3, 1]$, and $[4, 2, 1]$.

Assume that `buy_souvenirs` calls `transaction(2)`. Suppose the call returns $([2], 1)$, meaning that Amaru bought one souvenir of type 2 and the seller gave him back 1 coin. Observe that this allows us to deduce that $P = [4, 3, 1]$, since:

- For $P = [4, 3, 2]$, `transaction(2)` would have returned $([2], 0)$.
- For $P = [4, 2, 1]$, `transaction(2)` would have returned $([1], 0)$.

Then `buy_souvenirs` can call `transaction(3)`, which returns $([1], 0)$, meaning that Amaru bought one souvenir of type 1 and the seller gave him back 0 coins. So far, in total, he has bought one souvenir of type 1 and one souvenir of type 2.

Finally, `buy_souvenirs` can call `transaction(1)`, which returns $([2], 0)$, meaning that Amaru bought one souvenir of type 2. Note that we could have also used `transaction(2)` here. At this point, in total Amaru has one souvenir of type 1 and two souvenirs of type 2, as required.

Sample Grader

Input format:

```
N
P[0]  P[1] ... P[N-1]
```

Output format:

```
Q[0]  Q[1] ... Q[N-1]
```

Here $Q[i]$ is the number of souvenirs of type i bought in total for each i such that $0 \leq i < N$.